

Universidade do Minho
Licenciatura em Engenharia Informática
Laboratórios de Informática III

Relatório 2^a Fase do Projeto

Grupo 82

a107323 — Tomás Branco Dias
a106903 — Pedro Afonso Sousa dos Santos
a106800 — Eduardo Novo Araújo

Janeiro 2026

Conteúdo

1	Introdução	2
2	Metodologia utilizada e Descrição do sistema	3
2.1	Arquitetura Geral	3
2.2	Estrutura de dados	4
2.3	Modularidade e encapsulamento	5
3	Implementação das funcionalidades	6
3.1	Queries	6
3.1.1	Query 1 - Atualização	6
3.1.2	Query 2	6
3.1.3	Query 3	7
3.1.4	Query 4	7
3.1.5	Query 5	8
3.1.6	Query 6	8
3.2	Programa Interativo	8
4	Discussão	10
4.1	Gestão de memória	10
4.2	Programa de testes	10
4.3	Análise de desempenho	11
5	Conclusões	12
5.1	Análise Crítica e Resultados alcançados	12
5.2	Limitações	12
5.3	Otimizações e melhorias	13
6	Anexos	14
6.1	Tabela de desempenho	14

Capítulo 1

Introdução

Este relatório aborda a continuação do desenvolvimento do trabalho prático da disciplina de Laboratórios de Informática III, implementado em linguagem C, no âmbito do 2.^o ano da Licenciatura em Engenharia Informática.

O projeto tem como base um sistema centrado na gestão de aeroportos e voos, a partir do qual é explorado um conjunto de dados relativos a aeroportos, aviões, voos, passageiros, reservas e estatísticas de utilização do sistema. Os dados são carregados para estruturas de dados apropriadas e posteriormente utilizados para responder a um conjunto de *queries* previamente definidas. Pretende-se, assim, apresentar uma visão geral do contexto do projeto, bem como da abordagem e dos métodos adotados, recorrendo à aplicação prática de conceitos como modularização e encapsulamento.

Nesta segunda fase, a estrutura do projeto reforça a organização modular do sistema. Para garantir essa modularidade, o projeto encontra-se dividido em diversos módulos, cada um com responsabilidades bem definidas, tais como o *parsing* de dados, a execução de *queries* e a gestão das estruturas de dados. Esta abordagem visa facilitar a manutenção, a legibilidade e a evolução do sistema ao longo das diferentes fases do projeto.

Aliados a estes princípios, foram implementadas estratégias eficientes e utilizadas bibliotecas e ferramentas auxiliares, nomeadamente a biblioteca *GLib* e o *Valgrind*, com o objetivo de apoiar a gestão e análise da utilização de memória.

Capítulo 2

Metodologia utilizada e Descrição do sistema

2.1 Arquitetura Geral

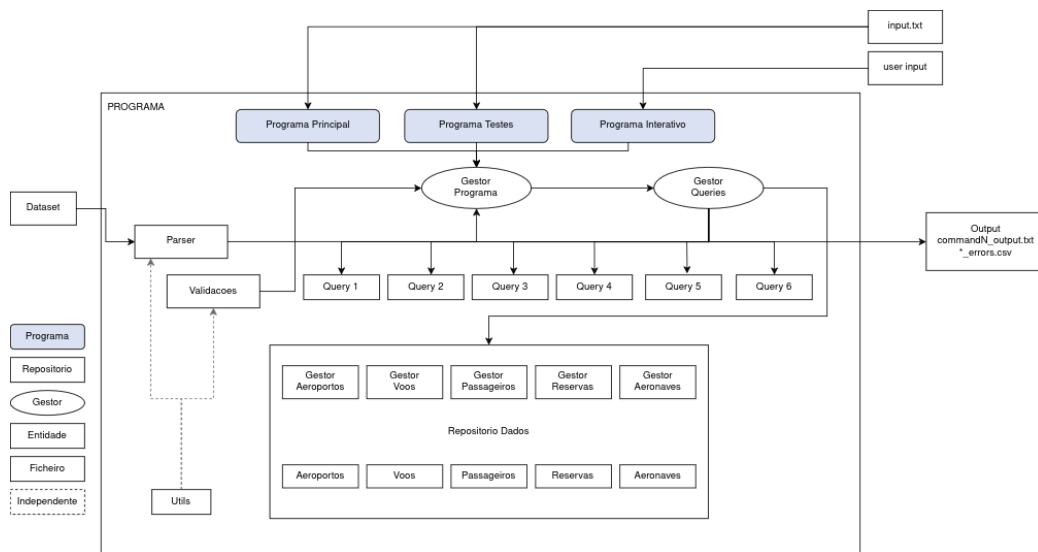


Figura 2.1: Arquitetura de referência para a aplicação a desenvolver

Tendo como base a arquitetura desenvolvida na primeira fase do projeto, a segunda fase introduziu um conjunto de melhorias com o objetivo de aumentar a modularidade, a clareza estrutural e a facilidade de manutenção do sistema. Procurou-se manter o fluxo principal de execução, mas redistribuindo responsabilidades por módulos mais bem definidos e independentes.

Na primeira arquitetura o programa-principal desempenhava um papel central, sendo responsável pela leitura do *dataset*, validação, *parsing*, gestão das entidades e execução das *queries*. Na segunda fase, estas responsabilidades foram reorganizadas através da introdução do Gestor de Programa, responsável pelo carregamento e coordenação global, e do Gestor de *Queries*, responsável por ler o ficheiro de comandos, identificar cada *query* e invocar a implementação correta.

A leitura e validação dos dados continuam a ser realizadas através dos módulos de *Parser* e *Validações*, mas passam a estar mais desacopladas do restante sistema, facilitando a sua reutilização e reduzindo dependências diretas. As linhas inválidas são encaminhadas para ficheiros de erros próprios, garantindo rastreabilidade e diagnósticos mais simples. Os dados das diferentes entidades (aeroportos, voos, passageiros, reservas e aeronaves) são agora centralizados em gestores específicos por entidade, que encapsulam o acesso a estruturas internas e expõem apenas operações de consulta e atualização.

Relativamente às *queries*, estas encontram-se organizadas em módulos independentes, o que permite a sua execução tanto no programa principal como no programa de testes ou no modo interativo. O Gestor de *Queries* atua como intermediário, coordenando a execução das *queries* e encaminhando o ficheiro de saída correspondente, garantindo uniformidade no tratamento dos comandos e evitando duplicação de lógica.

As saídas são geradas em ficheiros *resultados/commandX_output.txt*, enquanto as linhas inválidas de cada entidade são registadas em ficheiros de erros próprios, facilitando a validação e o diagnóstico. Por fim, o módulo de *Utils* mantém-se como um conjunto de funções auxiliares acessíveis transversalmente por todo o sistema, contribuindo para a reutilização de código e simplificação das implementações. Em síntese, a segunda fase do projeto representa uma evolução significativa da arquitetura inicial, preservando o fluxo lógico do programa, mas tornando-o mais modular, extensível e alinhado com boas práticas de desenvolvimento de software.

2.2 Estrutura de dados

Relativamente à estrutura de dados, não existem alterações significativas entre a Parte 1 e a Parte 2 ao nível das estruturas base utilizadas, mantendo-se o uso de *hash tables* e *arrays*. A principal diferença encontra-se na forma como estas são organizadas e acedidas.

As entidades principais são armazenadas em *hash tables* indexadas pelos

seus identificadores, permitindo acesso médio em tempo constante. Para suportar *queries* de forma eficiente, foram adicionadas estruturas auxiliares, como contadores agregados por companhia aérea, listas ordenadas para a *query* 5, tabelas de top 10 por semana e mapas de destinos por nacionalidade. Para alguns cenários temporais, são usadas contagens prefixadas por dia, evitando reprocessamento a cada execução da *query*. Além disso, foi adotada uma estratégia de *interning* de strings em voos para reduzir a duplicação de memória nos códigos de origem e destino.

2.3 Modularidade e encapsulamento

O encapsulamento tem como principal objetivo limitar o acesso direto aos dados ao módulo responsável pela sua gestão, promovendo maior facilidade de manutenção, robustez e flexibilidade do código. Esta prática permite uma separação bem definida entre a interface pública e a implementação interna, contribuindo para um sistema mais seguro, modular e de fácil manutenção.

Ao longo do projeto, o encapsulamento foi assegurado através da utilização de estruturas opacas e da criação de funções específicas para o acesso e atualização dos dados. Desta forma, qualquer operação relacionada com uma determinada entidade é realizada através das funções disponibilizadas pelo respetivo módulo, sem que os módulos externos necessitem de conhecer os detalhes da sua implementação. Esta abordagem permitiu que eventuais alterações a métodos fossem efetuadas de forma localizada, evitando modificações dispersas por diferentes partes do código. Assim, cada módulo mantém o controlo sobre os seus próprios dados e funcionalidades, expondo apenas as operações essenciais ao seu funcionamento externo.

Capítulo 3

Implementação das funcionalidades

3.1 Queries

As *queries* são processadas pelo Gestor de *Queries*, que interpreta o comando, valida os argumentos e invoca a função correspondente, escrevendo a saída no ficheiro associado. No modo interativo, as mesmas funções são reutilizadas diretamente, garantindo consistência entre modos de execução.

3.1.1 Query 1 - Atualização

Listar o resumo de um aeroporto, consoante o identificador recebido por argumento

A *query* 1 reutiliza a lógica já existente para obter a informação base do aeroporto e acrescenta o número de passageiros que chegaram e partiram. Estes valores são pré-computados após o carregamento, através da acumulação de passageiros por voo, e posteriormente agregados por aeroporto, evitando processamento extra durante a execução da *query*.

3.1.2 Query 2

Top N aviões com mais voos (com filtro opcional por fabricante)

Para responder a esta *query*, cada avião mantém um contador de voos incrementado durante a carga dos voos válidos. A consulta percorre todos

os aviões, aplica o filtro de fabricante quando indicado, ordena por número de voos (com desempate lexicográfico pelo identificador) e apresenta o top N com identificador, fabricante, modelo e contagem total.

3.1.3 Query 3

Aeroporto com mais partidas num período

Após o carregamento, o gestor de voos prepara uma estrutura de contagens prefixadas por dia para cada aeroporto de origem. Deste modo, a execução da *query* obtém o total de partidas num intervalo através de diferenças acumuladas, sem necessidade de percorrer todos os voos. Em caso de empate, o aeroporto com código lexicograficamente menor é selecionado e a saída inclui a sua informação descritiva.

3.1.4 Query 4

Qual o passageiro que esteve mais tempo no top 10 de passageiros que mais gastaram em viagens durante um período?

A implementação da *query* 4 baseia-se na exploração de informação previamente organizada pelo sistema. Para cada semana, o gestor de reservas mantém a lista dos dez passageiros com maiores gastos, evitando a necessidade de recalculá-los durante a execução da *query*.

O processamento recorre a uma tabela de dispersão, na qual cada entrada associa o número de documento do passageiro ao número de semanas em que este figura no top 10. Quando é definido um intervalo temporal, as datas são convertidas e depois mapeadas para identificadores de semana, permitindo focar apenas nas semanas pertinentes. Na ausência de um intervalo específico, todas as semanas disponíveis são consideradas.

Após a agregação dos dados, a tabela de dispersão é percorrida para determinar o passageiro com maior frequência no top 10. Em caso de empate, é selecionado aquele cujo número de documento é lexicograficamente inferior, assegurando a consistência do resultado. Por fim, os dados do passageiro são recuperados através do gestor de passageiros e apresentados no formato requerido, acompanhados do número total de semanas em que esteve presente no top 10.

3.1.5 Query 5

Top N companhias aéreas com mais tempo de atraso médio por voo

A *query* 5 recorre a dados previamente agregados e armazenados em *cache* pelo gestor de voos. Durante a carga, são atualizados o número de voos com atraso válido e o total de minutos de atraso por companhia. Na primeira execução, é gerada uma lista ordenada por atraso médio (com desempate lexicográfico) e reutilizada nas chamadas seguintes. A *query* seleciona as N companhias mais relevantes, apresentando o identificador, o número de voos e o atraso médio com três casas decimais.

3.1.6 Query 6

Listar o aeroporto de destino mais comum para passageiros de uma determinada nacionalidade

A *query* 6 recorre à informação previamente agregada pelo gestor de reservas, que mantém, para cada nacionalidade, uma tabela de contagem dos destinos escolhidos. Durante a carga e validação das reservas, os destinos de cada passageiro são contabilizados e posteriormente processados para identificar o destino com maior número de ocorrências, resolvendo empates de forma lexicográfica. A *query* adapta o separador de saída conforme o comando fornecido e apresenta o destino mais popular junto com o número de passageiros que o selecionaram. Esta abordagem, baseada em agregação pré-computada, garante eficiência e consistência na resposta, evitando a necessidade de percorrer todas as reservas a cada execução.

3.2 Programa Interativo

O programa implementa um sistema de interface interativa baseado em terminal, permitindo ao utilizador executar *queries* sobre os dados de aeroportos, aviões, voos, passageiros e reservas. A interação é realizada através de menus textuais e comandos digitados pelo utilizador.

O programa utiliza a estrutura *gestor_interativo_t* para gerir o estado geral da aplicação, incluindo os dados carregados e a ligação aos gestores específicos de cada entidade (*gestor_aeroportos_t*, *gestor_avioes_t*, *gestor_voos_t*, *gestor_passageiros_t* e *gestor_reservas_t*).

No início, o programa solicita ao utilizador o caminho do *dataset* e procede ao carregamento sequencial de todos os ficheiros relevantes. Durante este processo, são carregados e validados os dados de aeroportos, aviões, voos, passageiros e reservas, atualizando estatísticas e *caches* necessárias para execução eficiente das *queries*. Após o carregamento, o programa apresenta um menu textual, permitindo a seleção de *queries* e operações de análise de dados.

Cada *query* é associada a uma função específica que lê parâmetros do utilizador, valida entradas, executa a operação correspondente e apresenta o resultado formatado no terminal. O programa permite ainda personalização de cores através de códigos *ANSI*, melhorando a legibilidade e experiência do utilizador.

O sistema suporta também navegação simples pelo menu e execução contínua de *queries*, aguardando a interação do utilizador até à escolha de encerrar o programa. Esta abordagem textual garante simplicidade e compatibilidade com qualquer terminal sem necessidade de bibliotecas gráficas ou de manipulação avançada de janelas.

Capítulo 4

Discussão

4.1 Gestão de memória

Ao longo do desenvolvimento, foi dada especial atenção à gestão de memória, uma vez que o sistema processa grandes volumes de dados. Cada gestor de entidade possui funções de destruição responsáveis por libertar as estruturas internas. As *hash tables* e *arrays* são criadas com funções de destruição associadas, garantindo limpeza automática dos elementos. Adicionalmente, no módulo de voos foi aplicado *interning* de strings para reduzir duplicação e, após a atualização das estatísticas necessárias, alguns campos são descartados para minimizar a ocupação de memória.

Em paralelo, utilizou-se o *Valgrind* durante a fase de testes para identificar e corrigir fugas de memória e acessos inválidos. Esta validação contribuiu para uma implementação mais robusta e preparada para lidar com *datasets* de grande dimensão.

4.2 Programa de testes

Tal como na primeira fase, o programa de testes revelou-se indispensável para o bom desenvolvimento e validação do código, permitindo não só verificar a correção dos resultados das *queries*, como também acompanhar a evolução do desempenho do sistema. Através da comparação automática entre os ficheiros de *output* gerados e os resultados esperados, foi possível identificar discrepâncias de forma sistemática e garantir a exatidão das respostas produzidas.

Adicionalmente, o programa de testes executa o programa-principal e compara linha a linha cada ficheiro gerado com o respetivo esperado, registando o tempo total de execução e o pico de memória. A partir deste processo, são também calculados os tempos médios por *query*, usados na análise de desempenho. Esta informação revelou-se fundamental para detetar ineficiências, corrigir erros e assegurar que as alterações introduzidas na segunda fase não comprometeram a fiabilidade nem a eficiência do sistema, contribuindo assim para um desenvolvimento mais controlado e robusto.

4.3 Análise de desempenho

Para avaliar o desempenho, executámos o programa-testes nos *datasets* regular (*sem_erro*s) e large (*sem_erro*s_grande), com o ficheiro *inputs_fase2.txt* e os outputs esperados respetivos. Em cada ambiente, efetuámos uma execução completa (N=1) e registámos o pico de memória e os tempos médios por *query*. Os resultados detalhados encontram-se na Tabela 6.1 (Anexos).

No ambiente Linux, o tempo total do programa-principal foi de 3.73 s (regular) e 30.59 s (large), medido pelo programa-testes. De forma geral, os tempos médios mantêm-se baixos porque a maioria das *queries* utiliza informação pré-agregada (*caches*, contagens e top 10). O aumento observado no *dataset* large é consistente com o maior volume de dados processados e com o número de linhas geradas na saída. A memória apresenta variações reduzidas entre ambientes, indicando que o consumo é dominado pelas estruturas centrais carregadas em memória.

Capítulo 5

Conclusões

5.1 Análise Crítica e Resultados alcançados

Percebemos que um dos maiores desafios deste projeto residiu na análise e gestão da grande quantidade de dados disponibilizada. Acreditamos que, através da modularidade apresentada e do encapsulamento das diferentes responsabilidades, foi possível estruturar o código de forma eficiente, tornando-o mais organizado, fácil de analisar e simples de alterar ou estender. Consideramos, assim, que foram cumpridos todos os objetivos propostos para esta segunda fase do projeto.

Durante esta etapa, tivemos ainda a oportunidade de explorar mais aprofundadamente a biblioteca *GLib*, tirando partido das suas estruturas de dados e funcionalidades. Paralelamente, sentimos que conseguimos aprofundar o uso de ferramentas auxiliares e consolidar de forma mais consistente os conceitos de modularização e encapsulamento, fundamentais para o desenvolvimento de software robusto e escalável.

5.2 Limitações

Apesar da evolução arquitetural, o sistema mantém uma dependência significativa de memória, uma vez que toda a informação é carregada para estruturas internas e são criadas *caches* para acelerar algumas *queries*. Em *datasets* muito grandes, este modelo pode tornar-se menos escalável. Adicionalmente, o modo interativo permanece simples, não oferecendo funcionalidades como histórico de comandos ou exportação direta de resultados.

5.3 Otimizações e melhorias

Nesta fase, as otimizações focaram-se em reduzir recomputações e em deslocar trabalho para o momento de carregamento. Durante a leitura dos voos válidos, é atualizada a contagem de voos por avião e são agregados os atrasos por companhia; durante a leitura das reservas, é incrementado o número de passageiros por voo. No final do carregamento, estas contagens são propagadas para os aeroportos (chegadas e partidas), o que evita percursos adicionais sobre os dados a cada *query*.

Ao nível das *queries*, foram introduzidas estruturas auxiliares específicas. Para a Q3 é construída uma tabela de contagens prefixadas por dia e por origem, permitindo responder a intervalos através de diferenças acumuladas. Para a Q4, os gastos são agregados por semana e convertidos num top 10 semanal, eliminando a necessidade de recomputar o top a cada execução. Para a Q5, a lista de companhias é ordenada por atraso médio e guardada em *cache* na primeira execução, sendo reutilizada nas seguintes. Para a Q6, calcula-se o destino mais frequente por nacionalidade e mantém-se apenas o melhor destino e respetiva contagem, em vez de toda a tabela de destinos.

Na componente de memória, foi aplicado *interning* de strings para os códigos de origem e destino dos voos, reduzindo duplicações. Após a inserção de cada voo no gestor, campos que deixam de ser necessários (como *aircraft* e *airline*) são libertados, e várias tabelas auxiliares são destruídas depois de produzirem os resumos finais. Sempre que possível, foram usados ponteiros estáveis provenientes das entidades (por exemplo, o documento do passageiro) em vez de cópias, diminuindo alocações e melhorando a eficiência global.

Capítulo 6

Anexos

6.1 Tabela de desempenho

Dataset	Sistema Operativo	Processador	RAM	Memória	Query	Tempo (s)
Regular	Linux Ubuntu 24.04.1	i7-1165G7	16GB	19.4 MB	1	0.000020
					2	0.000020
					3	0.000020
					4	0.000010
					5	0.000020
					6	0.000010
Regular	Windows 11	Ryzen 7 5800H	16GB	20.1 MB	1	0.000024
					2	0.000024
					3	0.000024
					4	0.000012
					5	0.000024
					6	0.000012
Large	Linux Ubuntu 24.04.1	i7-1165G7	16GB	19.4 MB	1	0.000160
					2	0.000100
					3	0.000100
					4	0.000100
					5	0.000100
					6	0.000110
Large	Windows 11	Ryzen 7 5800H	16GB	20.6 MB	1	0.000192
					2	0.000120
					3	0.000120
					4	0.000120
					5	0.000120
					6	0.000132

Tabela 6.1: Desempenho do programa com os *datasets* regular e large.